

UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

SISTEM DE DETECTARE A INTRUZIUNILOR UTILIZÂND ÎNVĂȚAREA PRIN RECOMPENSĂ

Absolvent

Igescu Rareș-Andrei

Coordonator științific

Conf. univ. dr. Ciprian Păduraru

București, iunie 2026

Abstract

Lorem ipsum

Rezumat

Lorem ipsum

Cuprins

1	Introducere	4
1.1	Context și motivație	4
1.2	Contribuția lucrării	4
2	Fundamente teoretice și cercetări conexe	5
2.1	Sisteme de detecție bazate pe semnături (SIDS)	5
2.2	Sisteme de detecție bazate pe anomalii (AIDS)	6
2.3	Fundamentele Învățării prin Recompensă (RL)	7
2.3.1	Procese Decizionale Markov (MDP)	7
2.3.2	Ecuția Bellman și Deep Q-Networks (DQN)	8
2.4	Stadiul actual al cercetării	9
3	Dezvoltarea Sistemului de Detecție	10
3.1	Pregătirea și preprocesarea datelor experimentale	10
3.2	Proiectarea mediului de antrenament	11
3.2.1	Spațiul stărilor și spațiul acțiunilor	11
3.2.2	Proiectarea funcției de recompensă	12
3.2.3	Episoadele de antrenament și generalizarea	12
3.3	Algoritmul Deep Q-Network (DQN)	13
3.3.1	De la ecuația Bellman la funcția de pierdere	13
3.3.2	Experience Replay	14
3.3.3	Target Network	15
3.3.4	Politica de explorare ϵ -greedy	15
3.4	Arhitectura și Configurarea Agentului	16
3.5	Rezultatele Antrenamentului și Evaluarea Performanței	17
4	Testarea Sistemului în Timp Real	19
4.1	Arhitectura pipeline-ului	19
4.2	Alinierea caracteristicilor și citirea incrementală	20
4.3	Interfața de vizualizare	21
4.4	Rezultate și limitări	21
5	Concluzii	22

1. Introducere

1.1 Context și motivație

Rețelele informatice au evoluat în ultimele decenii, generând volume masive de trafic și complexitate sporită în identificarea amenințărilor cibernetice. Inteligența artificială și metodele de învățare prin Reinforcement Learning (RL) au demonstrat un potențial semnificativ în dezvoltarea sistemelor automate de detecție a intruziunilor (IDS). Majoritatea abordărilor tradiționale se concentrează pe aplicarea unor seturi de reguli statice, fără a ține cont de necesitatea adaptării dinamice la atacuri noi într-un mediu de rețea real.

Pentru a rezolva această problemă, paradigma rețelelor neuronale profunde de tip Deep Q-Network (DQN) devine tot mai relevantă. Un agent DQN poate evalua continuu starea rețelei, învățând autonom politici decizionale optime pentru a distinge comportamentele malițioase de cele legitime în timp real.

1.2 Contribuția lucrării

Prezenta lucrare de licență își propune dezvoltarea, implementarea și evaluarea unui Sistem de Detecție a Intruziunilor (IDS) adaptiv, bazat pe o arhitectură Deep Q-Network (DQN).

Pentru a atinge acest scop principal, au fost stabilite următoarele obiective specifice:

- **1. Modelarea mediului de rețea ca un Proces Decizional Markov (MDP):** Proiectarea unei funcții de recompensă personalizate care să permită agentului să găsească un echilibru optim între securitate și disponibilitatea rețelei.
- **2. Implementarea agentului RL:** Dezvoltarea arhitecturii DQN utilizând rețele neuronale profunde pentru aproximarea funcției de valoare, capabilă să proceseze spațiul continuu al caracteristicilor de trafic.
- **3. Antrenarea și validarea pe date moderne:** Utilizarea setului de date **CICIDS2017** (Canadian Institute for Cybersecurity), care conține trafic benign actualizat și cele mai comune familii de atacuri moderne, asigurând astfel relevanța practică a sistemului.

2. Fundamente teoretice și cercetări conexe

2.1 Sisteme de detecție bazate pe semnături (SIDS)

Sistemele de detecție bazate pe semnături reprezintă o metodă tradițională în securitatea rețelor, funcționând pe un principiu similar programelor antivirus clasice. Aceste sisteme monitorizează traficul de rețea și compară conținutul pachetelor cu o bază de date predefinită de ”semnături” (reguli) asociate atacurilor cunoscute.

Din punct de vedere algoritmic, eficiența acestui proces depinde de viteza cu care se efectuează potrivirea modelelor (*pattern matching*). Pentru a face față volumului mare de date în timp real, majoritatea soluțiilor consacrate utilizează algoritmul **Aho-Corasick** [1]. Acesta construiește un automat finit determinist din baza de date de semnături, permițând căutarea simultană a tuturor modelelor într-o singură parcurgere a pachetului de date, garantând astfel o complexitate liniară $\mathcal{O}(n)$ care este totodată independentă de numărul de semnături (k).

Semnăturile menționate anterior reprezintă, practic, amprenta digitală a unui atac, definită printr-un șir specific de bytes. Cel mai reprezentativ exemplu industrial este **Snort**, care utilizează reguli de forma:

Dacă pachetul vine de la IP-ul X și conține șirul hexazecimal Y , atunci generează o alertă.

Deși sistemele bazate pe semnături oferă o rată scăzută de alarme false (*False Positives*) pentru alarmele cunoscute, rigiditatea acestora constituie o mare breșă în contextul contemporan al securității:

- **Incapacitatea de a detecta atacuri Zero-Day:** Deoarece funcționează exclusiv pe baza unei liste negre (*blacklist*), orice atac nou apărut, pentru care nu a fost încă scrisă o semnătură, va trece neobservat.
- **Vulnerabilitate la Polimorfism:** Atacatorii pot modifica ușor semnătura unui malware (prin criptare sau schimbarea unor biți ne semnificativi) fără a-i altera funcționalitatea malițioasă. Această tehnică face ca semnătura din baza de date să nu se mai potrivească exact, păcălind algoritmul de pattern matching.
- **Mentenanță costisitoare:** Baza de date necesită actualizări manuale și continue, proces adesea prea lent față de viteza de propagare a noilor vectori de atac.

Aceste limitări necesită tranziția către paradigme de detecție euristice și comportamentale, precum

Sistemele Bazate pe Anomalii (AIDS), capabile să generalizeze și să identifice intenții malițioase necunoscute anterior.

2.2 Sisteme de detecție bazate pe anomalii (AIDS)

Acest model, propus inițial de **Denning** [2], reprezintă un sistem expert universal pentru detecția intruziunilor în timp real, bazat pe ipoteza că abuzurile informatice pot fi identificate prin monitorizarea anomaliilor din jurnalele de activitate. Prin utilizarea profilurilor statistice și a metricilor de comportament, sistemul învață interacțiunile normale dintre utilizatori și resurse pentru a semnala devierile suspecte, oferind un cadru adaptabil oricărei platforme, indiferent de tipul vulnerabilităților sau al atacurilor.

Pentru a putea detecta cu succes traficul suspect sau malițios, sistemul trebuie mai întâi să învețe acțiuni normale. Astfel, procesul de funcționare al unui sistem bazat pe anomalii poate fi împărțit în două:

- **Faza de antrenare:** Sistemul analizează un volum mare de trafic (protocoale utilizate des, lățime de bandă) care poate fi considerat normal pentru a stabili un profil de referință (cunoscut drept *baseline*) al activității.
- **Faza de detecție:** Orice deviație de la acest standard va fi marcată de sistem ca anomalie și va ridica o alertă la nivelul traficului.

Avantajul major al acestor sisteme este capacitatea de a detecta atacuri de tip **Zero-Day** deoarece sistemul nu are nevoie să cunoască de dinainte amprenta atacului; este suficient ca traficul malițios să se comporte diferit față de cel normal.

Totuși, implementarea clasică a sistemelor AIDS vine cu o provocare, anume rata ridicată a alarmelor false (*False-Positive Rate*). Rețelele moderne sunt dinamice, cu trafic variabil, iar un comportament atipic, dar legitim (trimiterea bruscă a mai multor fișiere de dimensiuni mari) poate fi marcat ca trafic malițios.

Pentru a rezolva această problemă a alarmelor false și a crea un sistem capabil să se adapteze continuu la dinamica rețelei, soluțiile moderne recurg la algoritmi avansați de inteligență artificială (AI). Dintre aceștia, o paradigmă care permite sistemului să învețe direct din consecințele deciziilor sale este **Învățarea prin Recompensă (Reinforcement Learning)**.

2.3 Fundamentele Învățăării prin Recompensă (RL)

Conform fundamentelor stabilite de **Sutton și Barto** [3], învățarea prin recompensă constă în capacitatea unui agent de a descoperi ce acțiuni trebuie întreprinse în anumite situații, cu scopul fundamental de a maximiza un semnal numeric de recompensă.

Multe sisteme reale nu au soluții perfecte, ci doar acțiuni mai bune în funcție de experiența acumulată până în acel moment. Un agent bazat pe învățarea prin recompensă încearcă să învețe cum să acționeze optim într-un mediu necunoscut. În contextul securității cibernetice, agentul joacă rolul sistemului de detecție, iar mediul din care se poate antrena este reprezentat de traficul rețelei monitorizate. Ciclul generic pentru un agent RL poate fi analizat în figura 2.1.

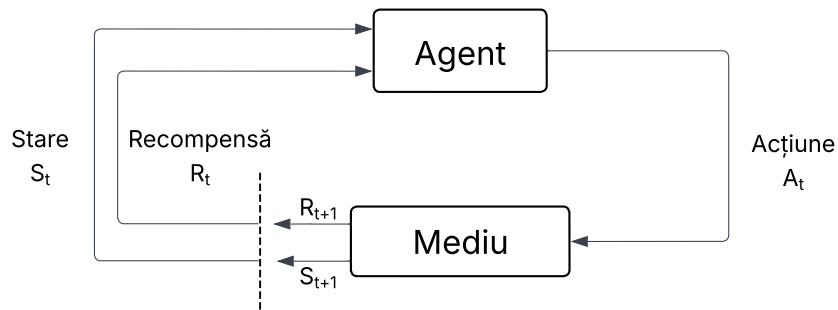


Figura 2.1: Ciclul de interacțiune Agent-Mediu în sistemul propus

2.3.1 Procese Decizionale Markov (MDP)

Procesele Markov sunt procese stochastice în care viitorul depinde doar de prezent. Mai precis, un proces Markov îndeplinește proprietatea Markov doar dacă probabilitatea de a ajunge într-o stare viitoare depinde exclusiv de starea curentă. Dacă spațiile de stări sunt finite, atunci acesta se numește proces de decizie Markov finit.

$$Pr(s_t | s_{t-1}, \dots, s_0) = Pr(s_t | s_{t-1})$$

Ecuția 2.1 Procese Markov de ordinul I

Această relație definește „independența de istoric”, sugerând că starea s_{t-1} încapsulează toate informațiile relevante din trecut necesare pentru a prezice starea viitoare s_t . În mod implicit, procesele

Markov se referă la procesele de ordinul I. Totuși, există situații complexe în care dependența se extinde asupra mai multor stări anterioare, ducând la procese de ordin superior:

$$Pr(s_t | s_{t-1}, \dots, s_0) = Pr(s_t | s_{t-1}, s_{t-2})$$

Ecuția 2.2 Procese Markov de ordinul II

Din puncte de vedere formal, un proces de decizie Markov este definit de următorul tuplu (S, A, P, R, γ) , unde:

- S reprezintă mulțimea finită de stări;
- A reprezintă mulțimea finită de acțiuni;
- $P(s' | s, a)$ este probabilitatea de a ajunge în starea s' după ce agentul alege acțiunea a în starea s ;
- $R(s, a, s')$ este recompensa primită pentru tranziția $(s \rightarrow s')$;
- $\gamma \in [0, 1]$ este factorul de discount, care controlează importanța viitorului.

În contextul detecției anomaliilor în trafic, proprietatea Markov presupune că pachetul curent (sau fereastra de trafic actuală) conține suficiente informații pentru a decide dacă este un atac, fără a fi necesară analiza întregului istoric al conexiunii de la inițializarea rețelei.

2.3.2 Ecuția Bellman și Deep Q-Networks (DQN)

Pentru a afla politica optimă, agentul analizează fiecare acțiune posibilă folosind funcția valoare-acțiune (*Action-Value*) $Q(s, a)$ care estimează recompensa așteptată pe termen lung. Actualizarea acestei funcții se face iterativ folosind Ecuția lui Bellman, care exprimă valoarea unei perechi stare-acțiune ca fiind suma dintre recompensa imediată și valoarea maximă estimată a stării următoare:

$$Q^*(s, a) = R(s, a, s') + \gamma \max_{a'} Q^*(s', a')$$

Ecuția 2.3 Ecuția Bellman

În varianta clasică, algoritmul Q-Learning memorează valorile lui Q într-un tabel. Această abordare a problemei funcționează atunci când spațiul stărilor este redus. Cu toate acestea, când vine vorba

de monitorizarea traficului unei rețele, numărul de stări este foarte vast, deoarece pachetele de date pot avea numeroase combinații de caracteristici. Astfel, utilizarea unei reprezentări tabelare este imposibilă în contextul prezentat.

Pentru a rezolva această problemă, **Mnih et al.** [4] au propus arhitectura Deep Q-Network (DQN). DQN elimină tabelul și utilizează o rețea neuronală profundă (cu ponderile θ) pentru a aproxima direct valorile optime: $Q(s, a; \theta) \approx Q^*(s, a)$.

În această abordare, caracteristicile traficului de rețea reprezintă datele de intrare (*input*) ale rețelei, iar ieșirile (*output*) sunt valorile Q estimate pentru acțiunile disponibile (Permite sau Blochează). Această generalizare permite sistemului IDS să identifice corect tipare de atac chiar și în configurații de trafic pe care nu le-a întâlnit în timpul antrenamentului.

2.4 Stadiul actual al cercetării

Utilizarea algoritmilor de Învățare prin Recompensă în securitatea cibernetică a cunoscut o creștere semnificativă, devenind o alternativă tot mai promițătoare la metodele clasice, cum ar fi Arborii de Decizie, care au nevoie de seturi de date corecte și complet etichetate și care totodată tind să își piardă eficiența în medii dinamice, unde tiparele de atac evoluează constant.

Studii relevante, cum sunt cele realizate de **Caminero et al.** [5] și **Lopez et al.** [6], au demonstrat că agenții RL pot fi utilizați cu succes pentru a detecta traficul malițios, modelând interacțiunile dintre atacator și sistemul de detectare a intruzinilor ca un joc secvențial. Această perspectivă permite adaptarea continuă a mecanismelor de detecție în funcție de comportamentul adversarului.

Totuși analiza literaturii actuale evidențiază două limitări majore:

1. **Utilizarea datelor învechite:** O mare parte din studiile existente încă își evaluează agenții de tip RL folosind seturi de date învechite, precum KDD99 sau NSL-KDD. Aceste informații nu mai reprezintă infrastructura modernă a unei rețele și nu includ tipologii actuale ale unui atac cibernetic, cum ar fi DDoS de tip Slowloris sau vulnerabilitățile specifice aplicațiilor web.
2. **Funcții de recompensă suboptime:** Multe implementări se concentrează exclusiv pe maximizarea ratei de detecție (Recall), ignorând penalizarea adecvată a alarmelor false (False Positives), ceea ce face sistemele inaplicabile într-un mediu de producție real.

3. Dezvoltarea Sistemului de Detecție

Pornind de la aceste fundamente teoretice, capitolul următor detaliază implementarea concretă a sistemului propus. Primul pas esențial în crearea sistemului inteligent de detecție a constat în organizarea setului de date. Pentru aceasta, a fost utilizat dataset-ul **CICIDS2017** [7], un standard contemporan pentru evaluarea sistemelor de securitate, care include atât trafic benign, cât și semănturile celor mai frecvente atacuri cibernetice (Brute Force, DDoS, Atacuri Web, etc.).

Manipularea și curățarea volumului masiv de date s-au realizat utilizând biblioteca *Pandas* [8] din limbajul Python.

3.1 Pregătirea și preprocesarea datelor experimentale

Etapa inițială de preprocesare a datelor a constat în eliminarea instanțelor invalide (de tip *Inf* și *NaN*) și în uniformizarea setului de date, demers esențial pentru a asigura convergența și stabilitatea procesului de antrenare a agentului. Pentru compatibilizarea datelor cu arhitectura unui model de decizie binară, eticheta primară a fost recodificată: traficul de rețea legitim a fost asociat cu valoarea **0** („Permite”), în timp ce toate instanțele de trafic malițios, indiferent de tipologia atacului, au fost agregate sub valoarea **1** („Blochează”). În faza finală, având în vedere discrepanțele majore de scală dintre parametrii de rețea (variind de la zeci de octeți la milioane de microsecunde), s-a impus aducerea tuturor variabilelor continue în intervalul standardizat $[0, 1]$. Această operațiune a fost implementată utilizând clasa *MinMaxScaler* din cadrul bibliotecii *scikit-learn* [9]. Etapa de normalizare previne riscul ca algoritmul de învățare să fie predominat de valorile absolute mari și se realizează prin aplicarea următoarei transformări matematice asupra fiecărei valori x dintr-o caracteristică:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Ecuția 3.1 Normalizarea datelor

Unde x_{min} și x_{max} reprezintă minimul, respectiv maximul caracteristicii respective în setul de date. Efectul vizual al acestei transformări este ilustrat în Figura 3.1, unde se poate observa cum distribuția originală a datelor este perfect conservată, însă transpusă integral în noul interval standardizat. Prin

parcursarea acestui flux, vectorul de stări obținut devine strict numeric și echilibrat dimensional, fiind complet pregătit pentru mediul de antrenament al agentului RL.

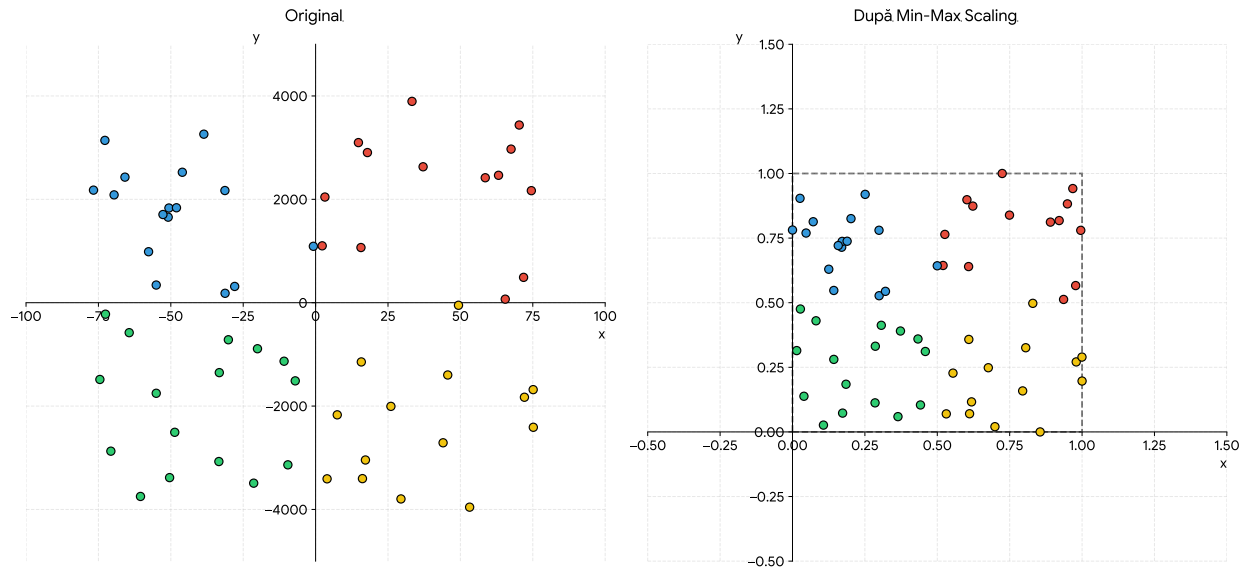


Figura 3.1: Exemplu de scalare a caracteristicilor

Pentru a asigura o evaluare obiectivă, setul de date a fost împărțit în proporție de 80%–20% în seturi de antrenament și testare, înainte de aplicarea normalizării. Scalarea a fost calibrată exclusiv pe datele de antrenament și aplicată ulterior setului de testare, prevenind astfel scurgerea statisticilor de testare în procesul de învățare.

3.2 Proiectarea mediului de antrenament

Odată încheiată preprocesarea datelor, pasul următor a constat în crearea unei punți de comunicare între agentul decizional și datele de trafic. Astfel, a fost definit un mediu de învățare personalizat (*Custom Environment*), având ca punct de plecare structura standard oferită de biblioteca *Stable-Baselines3* [10] și respectând rigorile impuse de API-ul *Gymnasium* [11]. Prin adaptarea riguroasă a acestui cadru de lucru la specificul setului de date, problema detecției atacurilor de rețea a putut fi modelată conceptual sub forma unui Proces Decizional Markov (MDP).

3.2.1 Spațiul stărilor și spațiul acțiunilor

Pentru ca rețeaua neuronală să poată procesa informația, a fost necesară definirea limitelor mediului:

- **Spațiul stărilor:** A fost definit ca un spațiu continuu de tip *Box* (o zonă mărginită în spațiul n -dimensional). La fiecare pas t , starea s_t reprezintă un vector ce conține caracteristicile unui

singur pachet de rețea. Datorită etapei anterioare de normalizare, s-a garantat că $s_t \in [0, 1]^N$, unde N este numărul de caracteristici (coloane) extrase din setul de date.

- **Spațiul acțiunilor:** Spațiul acțiunilor este discret, oferind agentului două decizii posibile la orice pas t : acțiunea $a_t = 0$ (permite pachetul) și acțiunea $a_t = 1$ (blochează pachetul).

3.2.2 Proiectarea funcției de recompensă

Nucleul mediului de antrenament, implementat în cadrul metodei $step()$, este reprezentat de funcția de recompensă $R(s_t, a_t)$. Aceasta a fost proiectată pentru a prioritiza detecția atacurilor, recunoscând că, în contextul securității cibernetice, omiterea unui atac real reprezintă un risc semnificativ mai mare decât generarea unei alarme false. Astfel, recompensa acordată agentului este guvernată de următoarea funcție matematică definită pe ramuri:

$$R(s_t, a_t) = \begin{cases} 1, & \text{dacă } a_t = y_t \text{ (Clasificare corectă)} \\ -2, & \text{dacă } a_t = 0 \text{ și } y_t = 1 \text{ (Atac ratat)} \\ -1.3, & \text{dacă } a_t = 1 \text{ și } y_t = 0 \text{ (Alarmă falsă)} \end{cases}$$

Ecuția 3.2 Recompensa agentului pe ramuri

Unde y_t reprezintă eticheta reală (cunoscută de mediu) a pachetului analizat la pasul t . Această alegere de design reflectă realitatea operațională a sistemelor de detecție a intruziunilor, unde consecințele unui atac nedetectat sunt, în general, mai severe decât ale unui fals pozitiv.

3.2.3 Episoadele de antrenament și generalizarea

Pentru a preveni fenomenul de *overfitting*, situație în care agentul memorează secvența specifică a pachetelor din setul de date, antrenamentul a fost structurat în episoade a câte 1000 de pași. O provocare suplimentară a fost dezechilibrul accentuat al claselor din setul de date CICIDS2017, în care aproximativ 83% din instanțe reprezintă trafic normal. Fără o intervenție explicită, agentul ar fi expus preponderent la trafic benign, dezvoltând o preferință sistematică pentru acțiunea „permite”, ceea ce ar conduce la o rată ridicată de atacuri neclasificate.

Pentru a contracara acest fenomen, metoda $reset()$ a fost adaptată să aplice o strategie de eșantionare echilibrată. La începutul fiecărui episod, cu o probabilitate de 50%, punctul de start este selectat aleatoriu din mulțimea instanțelor de atac, garantând că agentul este expus în mod echitabil atât la trafic normal, cât și la trafic malițios pe parcursul antrenamentului. Această inițializare stocastică

Algorithm 1 Logica de interacțiune și calculul recompensei

Require: $a_t \in \{0, 1\}$ ▷ Acțiunea primită de la agent (0 = Permite, 1 = Blochează)

- 1: $y_t \leftarrow$ eticheta reală a pachetului curent din setul de date
- 2: $r_t \leftarrow 0.0$ ▷ Inițializarea recompensei
- 3: **if** $a_t == y_t$ **then**
- 4: $r_t \leftarrow 1.0$ ▷ Agentul a clasificat corect
- 5: **else if** $a_t == 0$ **and** $y_t == 1$ **then**
- 6: $r_t \leftarrow -2.0$ ▷ Penalizare severă pentru atac ratat (Fals Negativ)
- 7: **else if** $a_t == 1$ **and** $y_t == 0$ **then**
- 8: $r_t \leftarrow -1.3$ ▷ Penalizare pentru alarmă falsă (Fals Pozitiv)
- 9: **end if**
- 10: $pas_curent \leftarrow pas_curent + 1$ ▷ Trecerea la următorul pachet de rețea
- 11: **if** $pas_curent \geq pasi_maximi$ **then**
- 12: $done \leftarrow \text{True}$ ▷ Episodul s-a încheiat
- 13: $obs_{t+1} \leftarrow \vec{0}$ ▷ Stare terminală (vector nul)
- 14: **else**
- 15: $done \leftarrow \text{False}$
- 16: $obs_{t+1} \leftarrow$ caracteristicile pachetului de la pas_curent
- 17: **end if**
- 18: **return** $obs_{t+1}, r_t, done$

controlată favorizează capacitatea de generalizare a agentului și reduce semnificativ rata atacurilor nedetectate.

3.3 Algoritmul Deep Q-Network (DQN)

În Capitolul 2 a fost introdusă ecuația lui Bellman, care definește valoarea optimă a unei perechi stare-acțiune ca suma dintre recompensa imediată și valoarea maximă estimată a stării următoare. Prezenta secțiune detaliază modul concret în care algoritmul DQN, propus de **Mnih et al.** [4], transformă acest fundament teoretic într-un proces de învățare funcțional, capabil să opereze în spațiul continuu și de dimensionalitate ridicată al caracteristicilor de trafic.

3.3.1 De la ecuația Bellman la funcția de pierdere

Algoritmul clasic Q-Learning actualizează iterativ o tabelă de valori $Q(s, a)$ pentru fiecare pereche stare-acțiune întâlnită. Însă când vine vorba de detecția intruziunilor, starea s_t este un vector continuu cu toate caracteristicile pachetului de rețea, ceea ce face imposibilă memorarea tuturor valorilor într-un tabel.

DQN rezolvă această problemă prin înlocuirea tabelii cu o rețea neuronală profundă parametri-

zată de ponderile θ , care aproximează funcția de valoare optimă: $Q(s, a; \theta) \approx Q^*(s, a)$. Rețeaua primește la intrare starea s_t și produce la ieșire câte o valoare Q estimată pentru fiecare acțiune disponibilă.

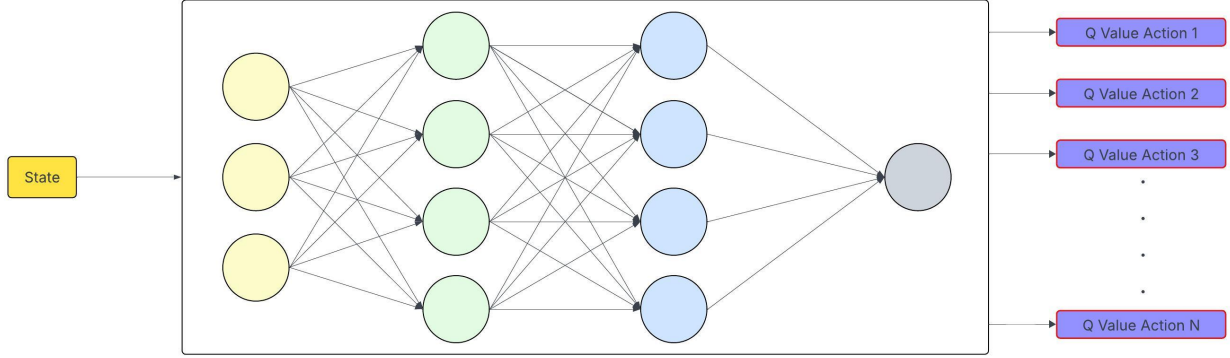


Figura 3.2: Structura Algoritmului DQN

Antrenarea rețelei se realizează prin minimizarea diferenței dintre valoarea Q prezisă de rețea și o valoare țintă derivată din ecuația Bellman. Această diferență, numită *Temporal-Difference Error*, este ridicată la pătrat pentru a forma funcția de pierdere:

$$\mathcal{L}(\theta) = E_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Ecuția 3.3 Funcția de pierdere a algoritmului DQN

Unde $\mathcal{D}$ reprezintă *replay buffer-ul*, iar θ^- sunt ponderile rețelei țintă, două mecanisme esențiale ale DQN care vor fi detaliate în subsecțiunile următoare. Minimizarea funcției $\mathcal{L}(\theta)$ se realizează prin *Stochastic Gradient Descent*, actualizând progresiv ponderile θ ale rețelei în direcția care reduce eroarea de predicție.

3.3.2 Experience Replay

DQN utilizează o memorie de reluare (*Experience Replay Buffer*) [?], notată $\mathcal{D}$. La fiecare pas t , agentul stochează tranziția (s_t, a_t, r_t, s_{t+1}) într-un buffer circular de dimensiune fixă, iar antrenamentul se realizează pe un *mini-batch* aleatoriu extras din $\mathcal{D}$. Astfel, tranzițiile dintr-un mini-batch provin din momente și episoade diferite, stabilizând procesul de învățare. Un beneficiu suplimentar este eficiența utilizării datelor: fiecare tranziție poate fi reutilizată de mai multe ori, ceea ce este deosebit de valoros pentru tipurile rare de trafic, cum ar fi un tip de atac slab reprezentat în setul de date

CICIDS2017. În configurația curentă, bufferul are o capacitate de 1.000.000 de tranziții, suficientă pentru a asigura diversitatea experiențelor pe parcursul celor 500.000 de pași de antrenament.

3.3.3 Target Network

Funcția de pierdere a DQN utilizează aceeași rețea atât pentru predicție, cât și pentru calculul țintei $r + \gamma \max_{a'} Q(s', a'; \theta)$. Dacă ambele sunt calculate cu aceleași ponderi θ , fiecare actualizare modifică simultan predicția și ținta, generând instabilitate în antrenament.

Soluția constă în introducerea unei *rețele țintă* cu ponderile θ^- , identică cu rețeaua principală, dar ale cărei ponderi sunt înghețate și sincronizate periodic la fiecare τ pași.

Între sincronizări, rețeaua țintă oferă valori stabile pentru calculul obiectivului de antrenare. În configurația sistemului propus, sincronizarea se realizează la fiecare $\tau = 10.000$ de pași.

3.3.4 Politica de explorare ε -greedy

Pentru a echilibra explorarea spațiului de acțiuni cu exploatarea cunoștințelor acumulate, agentul utilizează politica ε -greedy:

$$a_t = \begin{cases} \text{acțiune aleatorie din } A, & \text{cu probabilitatea } \varepsilon \\ \arg \max_a Q(s_t, a; \theta), & \text{cu probabilitatea } 1 - \varepsilon \end{cases}$$

Ecuția 3.4 Politica de selecție ε -greedy

Valoarea ε urmează un program de *descreștere liniară*, de la $\varepsilon_{init} = 1.0$ la $\varepsilon_{final} = 0.05$, pe parcursul primilor $T_{explorare} = 150.000$ de pași (30% din totalul antrenamentului):

$$\varepsilon(t) = \max \left(\varepsilon_{final}, \varepsilon_{init} - \frac{\varepsilon_{init} - \varepsilon_{final}}{T_{explorare}} \cdot t \right)$$

Ecuția 3.5 Descreșterea liniară a ratei de explorare

Această configurație permite agentului o fază inițială de explorare intensivă, în care descoperă diverse tipuri de trafic și atacuri din setul CICIDS2017, urmată de o fază de exploatare în care aplică politica învățată, menținând totuși un nivel minim de explorare de 5% pentru a putea reacționa la configurații rare de trafic.

3.4 Arhitectura și Configurarea Agentului

Rețeaua neuronală care aproximează funcția $Q(s, a; \theta)$ este un *Perceptron Multistrat* (MLP) [] cu următoarea structură:

- **Stratul de intrare:** Primește vectorul de stare $s_t \in [0, 1]^N$, unde N corespunde numărului de caracteristici numerice extrase din setul CICIDS2017.
- **Straturi ascunse:** Trei straturi cu dimensiunile [256, 256, 128] neuroni, fiecare aplicând o transformare liniară urmată de activarea ReLU:
- **Stratul de ieșire:** Produce două valori Q, corespunzând acțiunilor disponibile $Q(s_t, 0; \theta)$ (permite pachetul) și $Q(s_t, 1; \theta)$ (blochează pachetul):

$$a_t = \arg \max_{a \in \{0,1\}} Q(s_t, a; \theta)$$

Ecuția 3.6 Selecția acțiunii pe baza valorilor Q

Arhitectura [256, 256, 128] a fost aleasă deoarece topologia implicită din *Stable-Baselines3* (două straturi de 64 de neuroni) nu oferea suficientă capacitate de reprezentare pentru dimensionalitatea ridicată a spațiului de observație al setului CICIDS2017.

Tabelul 3.1 centralizează toți hiperparametrii agentului DQN, detaliați în secțiunile anterioare.

Parametru	Valoare	Justificare
Algoritm	DQN	Eficient pentru acțiuni discrete
Politică	MlpPolicy	Spațiu de observație continuu
Arhitectură rețea	[256, 256, 128]	Dimensionalitate ridicată a intrărilor
Rată de învățare (α)	0.001	Standard pentru DQN
Factor de discount (γ)	0.95	Convergență stabilă pe termen mediu
Fracțiune explorare	0.30	Explorare extinsă în faza inițială
ϵ_{final}	0.05	Menținerea unui nivel minim de explorare
Dimensiune replay buffer	1.000.000	Diversitate ridicată a experiențelor
Frecvență sincronizare θ^-	10.000 pași	Stabilitatea țințelor de antrenare
Pași de antrenament	500.000	Convergență completă a politicii
Pași per episod	1.000	Varietate maximă a instanțelor văzute

Tabela 3.1: Hiperparametrii agentului DQN

3.5 Rezultatele Antrenamentului și Evaluarea Performanței

După finalizarea procesului de antrenament pe parcursul a 500.000 de pași, agentul DQN a fost evaluat pe un set de testare de 20% din datele CICIDS2017, complet separat de datele folosite la antrenament. Figura 3.3 ilustrează evoluția acurateței de-a lungul antrenamentului și matricea de confuzie obținută în urma evaluării finale. Rezultatele confirmă că agentul a învățat să clasifice eficient traficul de rețea.

Absența unor scăderi bruște în performanță pe parcursul antrenamentului indică faptul că mecanismul de eșantionare echilibrată și fixarea generatorului de numere aleatoare ($seed = 42$) au asigurat o distribuție consistentă a experiențelor pe durata antrenamentului.

Agentul a obținut o acuratețe globală de 98.2% pe instanțele din setul de testare. Din totalul de 85.139 de atacuri, au fost detectate corect 81.827. Astfel, rata de detecție a atacurilor a ajuns la 96.1%. În ceea ce privește traficul legitim, din totalul instanțelor normale, doar 5.547 au generat alarme false, cu o rată de 1.3%.

Comparativ cu o configurație anterioară în care penalizarea alarmelor false era -1.0 , creșterea acesteia la -1.3 a redus numărul de False Positive-uri de la 9.591 la 5.547 (o reducere de 42.2%), cu costul unei ușoare creșteri a False Negative-urilor de la 2.283 la 3.312. Această dinamică confirmă compromisul fundamental al oricărui sistem de detecție (*Precision–Recall trade-off*): reducerea alarmelor false implică în mod inevitabil o ușoară scădere a ratei de detecție.

Performanțele obținute sunt competitive cu abordările clasice de Machine Learning aplicate pe același set de date CICIDS2017, unde metodele precum Random Forest sau SVM raportează acurateți în intervalul 95 – 99% [7].

Totuși, sistemul propus prezintă câteva limitări inerente abordării adoptate. Clasificarea este binară, mai precis, agentul distinge doar între trafic normal și atac, fără a identifica tipul specific al atacului. Totodată, atacuri mai recente care nu se găsesc în setul de date CICIDS2017 pot trece neobservate, astfel performanța pe tipuri de atacuri mai noi rămâne o direcție deschisă de cercetare.

DQN Intrusion Detection System — Training & Evaluation Report

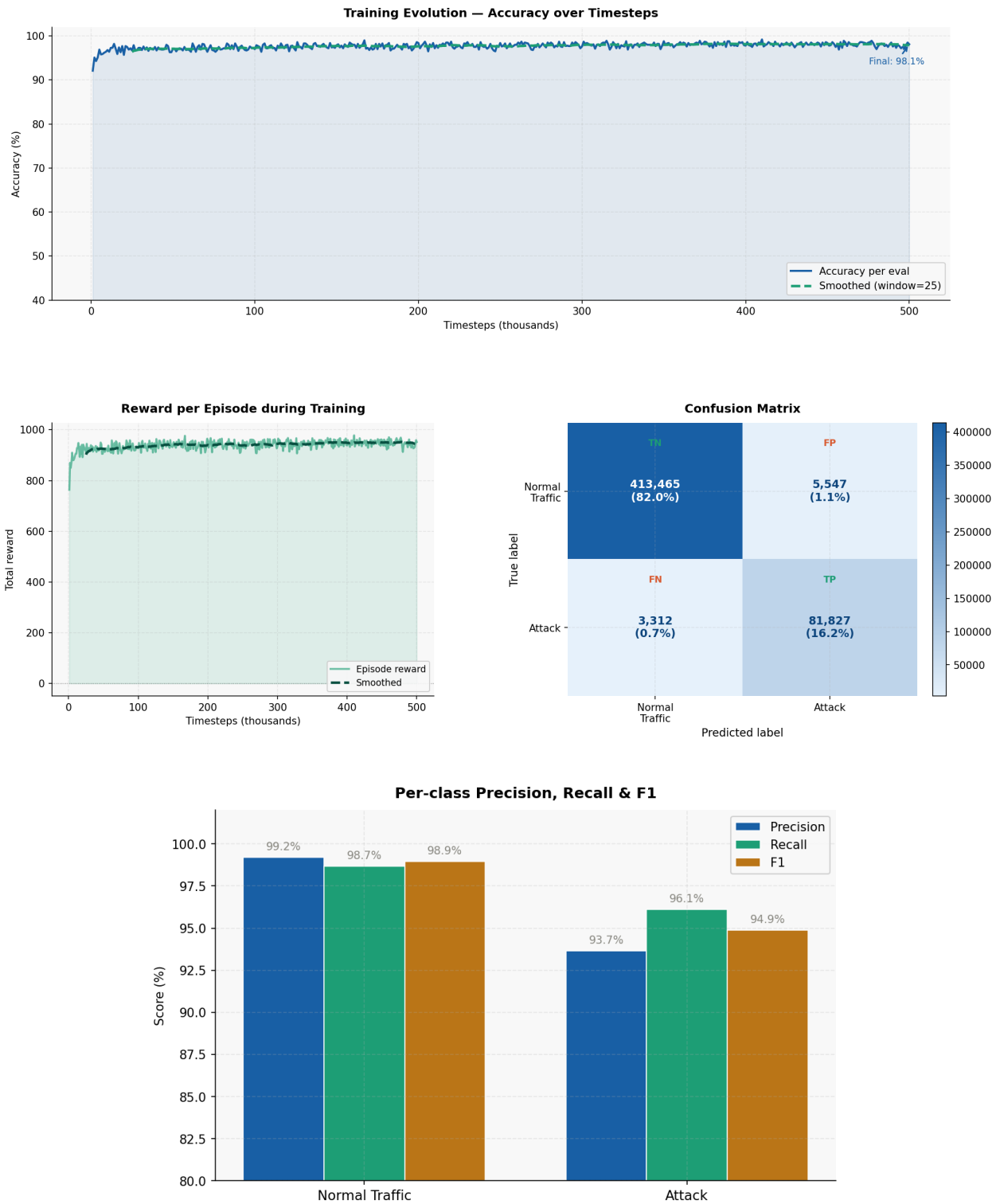


Figura 3.3: Rezultatele evaluării agentului DQN

4. Testarea Sistemului în Timp Real

Validarea unui sistem de detecție exclusiv pe un set de date static oferă doar o imagine parțială a performanței. Pipeline-ul complet de captare, extragere a caracteristicilor și inferență introduce provocări suplimentare care nu apar în antrenament: latența dintre un pachet și disponibilitatea fluxului pentru clasificare, alinierea caracteristicilor extrase în timp real cu schema folosită la antrenament și robustețea sistemului în fața traficului real, eterogen și neetichetat. Capitolul de față descrie arhitectura pipeline-ului dezvoltat pentru testarea agentului DQN în condiții reale, împreună cu rezultatele obținute pe o sesiune de captare de trafic legitim.

4.1 Arhitectura pipeline-ului

Pipeline-ul urmează o arhitectură în trei etape decuplate: (1) captarea pachetelor la nivelul interfeței fizice prin biblioteca *Npcap*; (2) extragerea caracteristicilor prin instrumentul *CICFlowMeter* [?], care grupează pachetele în fluxuri bidirecționale și calculează cele aproximativ 80 de caracteristici statistice ce stau la baza setului CICIDS2017; (3) clasificarea și raportarea printr-un script Python (denumit *detectorul*) care monitorizează fișierul CSV produs de *CICFlowMeter*, aliniază caracteristicile la schema de antrenament, aplică normalizarea *MinMax* fixată anterior și invocă agentul DQN. Alegerea *CICFlowMeter* nu este întâmplătoare: fiind instrumentul original utilizat la generarea setului de antrenament, se garantează coerența semantică între caracteristicile văzute la antrenament și cele văzute în inferență.

Comunicarea între componente se realizează exclusiv prin fișierul CSV intermediar, decuplare care permite înlocuirea oricăreia dintre cele două componente fără a afecta cealaltă și simplifică investigarea eventualelor probleme. Arhitectura este ilustrată în Figura 4.1.

Extragere flux

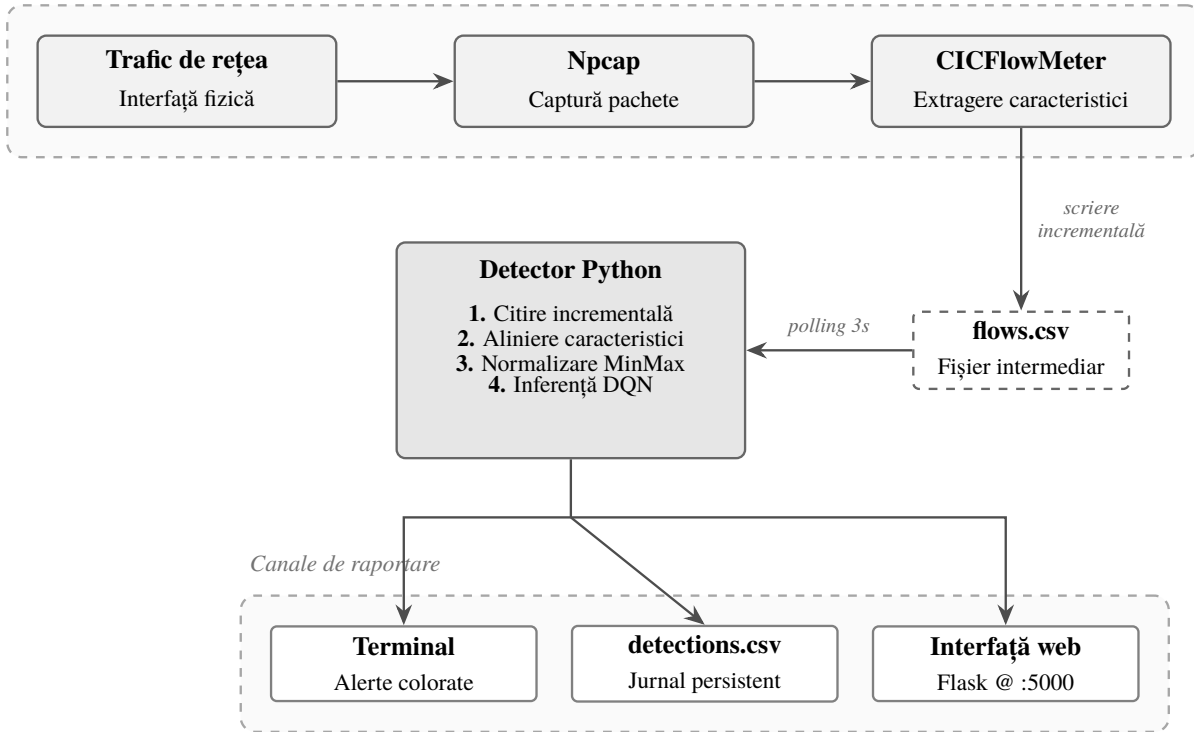


Figura 4.1: Arhitectura pipeline-ului de testare în timp real

4.2 Alinierea caracteristicilor și citirea incrementală

Cea mai critică problemă a pipeline-ului constă în alinierea caracteristicilor dintre cele două surse. Varianta contemporană a CICFlowMeter emite 80 de caracteristici cu denumiri *snake_case*, pe când setul CICIDS2017 folosește 52 de caracteristici cu denumiri în *Title Case*. Orice dezacord poate conduce la o aliniere incorectă silențioasă, în care modelul primește caracteristicile într-o ordine greșită și produce predicții plauzibile, dar complet aleatoare. Pentru a preveni acest scenariu, detectorul menține o listă canonică a celor 52 de caracteristici în ordinea exactă; la fiecare flux, denumirile sunt traduse prin alias-uri predefinite, coloanele neutilizate sunt ignorate, iar caracteristicile lipsă sunt completate cu zero — comportament identic cu cel aplicat la antrenament.

CICFlowMeter scrie fluxurile incremental, pe măsură ce acestea se închid prin steagurile TCP *FIN/RST* sau prin expirarea timeout-ului de inactivitate (120 secunde). Detectorul monitorizează fișierul printr-o buclă de interogare la intervale de 3 secunde, menținând un offset în octeți pentru fiecare fișier. Operarea în mod binar garantează corectitudinea offset-urilor indiferent de codificarea fișierului, iar pentru a evita parsarea liniilor parțiale aflate în scriere, detectorul avansează offset-ul

doar până la ultimul caracter complet de sfârșit de linie.

4.3 Interfața de vizualizare

Detectorul expune o interfață web pe portul local 5000, servită prin *Flask*. Interfața prezintă patru indicatori agregați (numărul total de fluxuri, fluxurile normale, atacurile detectate, rata atacurilor) și o tabelă cu ultimele 50 de fluxuri marcate cromatic după verdict, actualizată la fiecare două secunde prin apeluri asincrone. Rolul interfeței este dublu: permite inspectarea rapidă a comportamentului agentului în timp real și servește ca instrument demonstrativ pentru validarea academică.

4.4 Rezultate și limitări

Pentru evaluare, a fost efectuată o sesiune de captare pe o mașină Windows într-o rețea domestică tipică, constând în navigare web obișnuită, fără activitate malițioasă intenționată, pentru a măsura direct rata alarmelor false (*False Positive Rate*) în absența atacurilor reale. Pe durata sesiunii au fost clasificate 1000 de fluxuri bidirecționale: 999 trafic legitim și 1 atac, corespunzând unei rate a alarmelor false de 0.1% pe trafic real. Rezultatul este consistent cu performanța măsurată pe setul de testare la antrenament (1.3%) și confirmă că performanța offline a agentului se transferă în condiții de producție. Unicul flux clasificat drept atac corespundea unei conexiuni de pe portul 443 al unui server extern către un port efemer al mașinii de test, prezentând asimetrii statistice care pot fi confundate cu tipare de răspuns ale unor atacuri DDoS reflexive — probabil un fals pozitiv legitim, reflectând o limitare intrinsecă a oricărui sistem care operează exclusiv la nivel de flux.

Rezultatele trebuie interpretate în contextul câtorva limitări. CICFlowMeter raportează un flux doar la închidere, introducând o latență de câteva secunde până la peste un minut între atac și detecție. Modelul tratează fiecare flux independent, fără context între fluxuri consecutive, ceea ce îl face incapabil să detecteze atacuri manifestate doar la nivel agregat (de exemplu, un *port scan* distribuit temporal). Clasificarea binară nu permite identificarea tipologiei atacului, o extindere naturală fiind trecerea la o formulare multi-clasă.

Capitolul de față a demonstrat că agentul DQN antrenat pe CICIDS2017 poate fi integrat într-un pipeline funcțional de detecție în timp real și produce clasificări consistente pe trafic capturat live. Evaluarea ratei de detecție pe atacuri reale, într-un mediu controlat, rămâne o extensie naturală și reprezintă direcția prioritară pentru continuarea acestei cercetări.

5. Concluzii

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean dignissim metus justo, nec pharetra mauris tincidunt id.

6. Bibliografie

- [1] A. V. Aho and M. J. Corasick, “Efficient string matching: an aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [2] D. E. Denning, “An intrusion-detection model,” *IEEE Transactions on software engineering*, no. 2, pp. 222–232, 1987.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, second ed., 2018.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] G. Caminero, M. Lopez-Martin, and B. Carro, “Adversarial environment reinforcement learning algorithm for intrusion detection,” *Computer Networks*, vol. 159, pp. 96–109, 2019.
- [6] M. Lopez-Martin, B. Carro, and A. Sanchez-Esguevillas, “Network intrusion detection based on reinforcement learning and deep generative models,” *IEEE Access*, vol. 8, pp. 151474–151486, 2020.
- [7] Canadian Institute for Cybersecurity (CIC), “Intrusion detection evaluation dataset (cic-ids2017).” <https://www.unb.ca/cic/datasets/ids-2017.html>, 2017. Universitatea din New Brunswick (UNB). [Accesat la: 18.02.2026].
- [8] The pandas development team, “pandas-dev/pandas: Pandas.” <https://doi.org/10.5281/zenodo.18675244>, Februarie 2020. Versiunea 3.0.1. DOI: 10.5281/zenodo.18675244 [Accesat la: 25.02.2026].
- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

- [11] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.